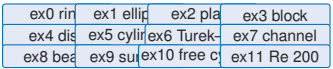
TOUR OF IBAMR'S OWN IBFE EXAMPLES

`IBFE/explicit/ex0` to `ex11` and the two complex-fluid examples with bodies: what each one is, what it tests, how it is built and run today, what came out when all of them were run for this chapter, and which ones no longer run as shipped



`examples/IBFE/explicit/`, IBAMR 0.19.0, all built and run for this chapter

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part IV • Chapter 25

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. say, for each of IBAMR's twelve finite-element examples, what body is in it, what drives it, what it measures, and which published test it reproduces;
2. build any of them outside IBAMR's tree with the six-line `CMakeLists.txt` and run it on a laptop, and know which ones are smoke tests as shipped (one hundred steps) and which are benchmarks of half an hour to half a day;
3. recognize the four mechanisms they add to Part IV's: a surface mesh with its own method, a tether that reads fields from another mesh, rigid-body motion by direct forcing, and a stabilized nearly incompressible beam;
4. read the `.curve` files they write (volume, drag and lift, a tip's position, a centre's path) and the MATLAB script that turns lift into a Strouhal number;
5. know which examples stop as shipped with IBAMR 0.19.0 and why (`ex9` on a mesh-dimension assertion; `ex5` and `ex6` on the "time step size change" abort), what that abort means, and what makes the two of them run;
6. pick the example to start from for a problem of your own, and the chapter of this series that explains its mechanism.

> Chapter 15 toured the point-method examples and ran forty of them; this chapter does the same for the finite-element ones, which chapter 15's map had wrong (slide 7).

WHERE THIS CHAPTER COMES FROM

- "6 – IBFE – What do Boyce's examples do?" (2020; 19 slides; `IBFE_Lecture_6`): a tips slide (Makefile and script per folder, "timestep size change encountered", the convergence testers), one slide for each of `ex0`–`ex8`, `ex10` and `ex11` with the lecturer's reading of what it does ("I believe", "I think" in several), three figure crops from Griffith and Luo (2017) for `ex1`, `ex4`, `ex5`, and two slides on the complex-fluids examples ("I could not get the examples with immersed boundaries to work (yet)"). No slide for `ex9`.
- The 2013 lists: the *IBFE-VisIt* notes and the MATH 769 IBFE deck listed the examples of the day (then `ex0`–`ex5`); chapter 17 quoted the 2014 getting-started deck's list.
- IBAMR 0.19.0: the twelve `example.cpp` files and their inputs, the READMEs of `ex5`, `ex10` and `ex11`, and `complex_fluids/ex2` and `ex4`; `IBFEMethod.cpp` for the assertion that stops two of them.
- The runs: every example built standalone against 0.19.0 and run on four cores, as shipped and, where the shipped input is a hundred-step smoke test, twenty times longer; the 3D inputs; the two complex-fluid examples with bodies. Report lines in `wsl/final/run_ibfe_tour.sh`'s output; pictures by `wsl/final/visit_ibfe_tour.sh`.
- New here: the identification of each example from its code (`ex6` is the Turek–Hron benchmark, `ex8` a clamped beam over a slot, `ex2` not porous); the run table; the three examples that do not run as shipped, why, and what makes two of them run; the complex-fluids examples' one-rank condition; the correction of chapter 15's map; the mechanisms table; own renders in place of the figure crops.

THE MAP (1): EX0 TO EX5

	the body and what drives it	what it tests or measures	chapter here
ex0	an elastic ring (radius 0.25, thickness 0.0625) at rest in a periodic 1×1 box, with the exact pressure jump set as the initial pressure	a static equilibrium: the fluid should stay at rest and the ring's area constant (`volume.curve`); the "sharp" and "smooth" pressure cases	17 (the mapping), 19
ex1	the same ring started as an ellipse ($\gamma = 0.15$) in a fluid of viscosity 10^{-2}	the dynamic shell test of Griffith and Luo, their Fig. 4, $0 \le t \le 0.75$	19
ex2	a soft plate ($\mu = 10$, 0.1 wide, full height) across a 2×1 box; normal stress -0.1 on the left face and $+0.1$ on the right	a body pushed by a traction difference with no tether; also in 3D	22 (boundary conditions)
ex3	a soft block ($c_1 = 0.05$) on the floor of a 2×2 cavity whose lid moves at 0.5	a solid deformed by a shear flow; the lid velocity is smoothed to zero at the corners	19
ex4	a soft disc (radius 0.2, $c_1 = 0.05$) at $(0.6, 0.5)$ in a 1×1 lid-driven cavity, lid speed 1	Griffith and Luo's Fig. 10 ($3 \le t \le 7$); `volume.curve`; a stabilization β scaled with the grid; also in 3D	19
ex5	a cylinder (radius 0.5) in a 16×16 box at $\mathrm{Re} = 200$, inflow ramped by $\tanh$ with a decaying transverse perturbation	vortex shedding, C_D and C_L (their Fig. 12); as a surface mesh with `IBFESurfaceMethod` or as a solid disc with body tethers; also in 3D; aborts as shipped (slide 24)	21, 23

THE MAP (2A): EX6 TO EX8

	the body and what drives it	what it tests or measures	chapter here
ex6	a rigid disc (radius 0.05 at $(0.2, 0.2)$) with an elastic beam (0.35×0.02) behind it in a 2.46×0.41 channel; parabolic inflow of mean 2 ramped over 2 s; $\rho = 10^3$, $\nu = 10^{-3}$, $\mu_s = 2 \times 10^6$, $\nu_s = 0.4$	the Turek–Hron FSI3 benchmark: the beam tip's path (`A_x.curve`, `A_y.curve`), C_D, C_L; aborts as shipped (slide 24)	21
ex7	the collapsible channel	chapter 23 in full	23
ex8	two rigid blocks with a slot between them and a thin elastic beam (1×0.032) clamped over the slot; a pressure of 80, ramped over 2 s, pushes fluid up through the slot	the beam's deformed length and the flux through the slot; a nearly incompressible beam with a grid-scaled stabilization; Chebyshev-spaced beam nodes	19, 21

THE MAP (2B): EX9 TO EX11

	the body and what drives it	what it tests or measures	chapter here
ex9	a cylinder's *boundary* mesh tethered, in position and velocity, to fields on the solid mesh (at rest as shipped) in a tall still box	a template for a surface tethered to a prescribed motion; stops at once in 0.19.0 (slide 23)	21
ex10	a neutrally buoyant rigid cylinder (diameter 1) free to move in a plane Couette flow (walls at ± 1, channel 4 high, 160 long, periodic) at $\mathrm{Re} = 40$	inertial migration of a free body; the centre's path and velocity (`X.curve`, `U.curve`); direct forcing of a rigid body	this chapter
ex11	a fixed rigid cylinder (radius 0.5) at $\mathrm{Re} = 200$ in a 32×16 box, ramped inflow with a perturbation	C_D, C_L and the Strouhal number (`compute_St.m`); direct forcing	this chapter

THE MAP (3): WHAT CHAPTER 15 SAID, AND THE CORRECTION

- Chapter 15's slide 31 ("`IBFE/explicit`: twelve examples for Part IV") gave a one-line map that was wrong: it listed this series' own examples (the rotated plate, the active tension, the plate that hits the disc, the flow tank, the scalar) under IBAMR's numbers. The slide is corrected in the current build to the map of slides 4–6; if your copy says "ex6 a contracting body with an active stress", it is the old one.
- The 2020 lecture's own readings, checked against the code: `ex0` "tests the conservation of volume in the shell" (right: `volume.curve`); `ex2` "porous FE boundaries . . . the first example" (no: a soft plate pushed by a traction difference; there is no porosity in the code); `ex6` "I believe it comes from a paper by Boyce, but I have not found it yet" (it is Turek and Hron's 2006 benchmark, the FSI3 parameters); `ex7` "I believe this is an example of peristaltic transport" (a collapsible channel, chapter 23); `ex8` "two solid blocks with a rigid, porous plate on top" (an elastic beam clamped over a slot, not rigid, not porous).
- "I believe" and "I think" were the right words in 2020; the code and the runs replace them here.

BUILDING AND RUNNING THEM (1): 2020

2020 (slide 3, “Tips for running examples”): “Remember to move a Makefile and script.sh to each folder. Make sure the Makefile has the right dimensions, make sure the script.sh uses the right input file. Some of the examples go unstable (timestep size change encountered). Just reduce DT in the input file and rerun. Some of the examples have other .C or .cpp files for convergence tests, etc. The Makefile we use compiles all .C files in the folder. Drop the .C / .cpp and then make.”

- Every point survives in a new form: the dimension is the CMake target (`IBAMR2d` or `3d`), the input file is the argument, the time-step abort is slide 24 (where “just reduce DT” is tried on the two examples that abort, and is not enough), and the extra files are the ones the CMake list leaves out.
- The 2020 course ran them in IBAMR’s tree with the course Makefile (chapter 15’s first route). The examples are the same twelve today, with the same inputs; the tip about instability was written because two of them aborted then as they do now.

BUILDING AND RUNNING THEM (2): NOW

Chapter 15's second route; `wsl/final/run_ibfe_tour.sh` does this for all of them:

```
cp -r $IBAMR_SRC/examples/IBFE/explicit/ex4 ex4 && cd ex4
rm CMakeLists.txt Makefile.am Makefile.in     # the in-tree build files
cat > CMakeLists.txt <<EOF
cmake_minimum_required(VERSION 3.15...3.30)
project(ex4 LANGUAGES C CXX)
find_package(IBAMR 0.16.0 REQUIRED)
add_executable(main2d example.cpp)           # not convergence_tester.cpp
target_link_libraries(main2d PRIVATE IBAMR::IBAMR2d)
EOF
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j 4
mpirun -np 4 ./build/main2d input2d          # or main3d input3d
```

- `ex4` and `ex5` carry a `convergence_tester.cpp`; `ex7` three inputs (`caseC`, `caseS2`, `caseSP`); `ex2`, `ex4`, `ex5`, `ex9` an `input3d` as well. Build time is 30–50 s each on the laptop.

BUILDING AND RUNNING THEM (3): HOW LONG THEY ARE

- Smoke tests as shipped. `ex0`–`ex6` and `ex8` have `END_TIME = 100*DT`: one hundred steps, a few seconds, enough to see that the program builds and the first frames look right, not enough to see the physics. `ex7` has 1000 steps. The run table (slide 28) gives each one's hundred steps in physical time.
- Benchmarks. `ex9` runs to $t = 15$, `ex11` to 130 (65 000 steps on a 1024×512 fine grid), `ex10` to 1200 (240 000 steps): "This is a long simulation with many file writes" (2020, slides 15–16). These are the ones with `.curve` output meant to be compared with published numbers.
- For the pictures of this chapter, the smoke tests were rerun twenty times longer (`END_TIME = 2000*DT`) under a fifteen-minute cap, and the benchmarks for half an hour each; the tables say how far each got: `ex11` to the end of its 130 time units, `ex10` to 52 of 1200. Chapter 16's rule applies before any of them is trusted: refine and compare.
- The three Griffith and Luo figures the 2020 deck cropped (`ex1`'s shell, `ex4`'s disc, `ex5`'s cylinder) are replaced here by renders of the runs (slides 31–32); the paper is the reference for the numbers.

EX0 AND EX1 (1): THE RING'S CODE

```
static const double R = 0.25;        // the ring's inner radius
static const double w = 0.0625;      // its thickness
static const double gamma = 0.0;     // ex1: 0.15, an ellipse
static const double mu = 1.0;
void coordinate_mapping_function(libMesh::Point& X, const libMesh::Point
  & s, void* /*ctx*/)
{
    X(0) = (R + s(1)) * cos(s(0) / R) + 0.5;             // s(0): arc
  length round the ring
    X(1) = (R + gamma + s(1)) * sin(s(0) / R) + 0.5;   // s(1): across
  the thickness
}
void PK1_stress_function(...) { PP = (mu / w) * FF; ... }
```

- The mesh is a rectangle ($2\pi R$ by w, `QUAD9`, `MFAC = 4`) that chapter 17's coordinate mapping bends into a ring: s_0 is the arc length, s_1 the position across the thickness.
- The stress $\mu\mathbb{F}/w$ is a membrane tension spread over the thickness: the simplest stress of chapter 19, with no resistance to compression or shear beyond the fluid's.
- One file serves both examples: `gamma` is the only difference, and the input's `MU` and viscosity change with it.

EX0 AND EX1 (2): THE TEST AT REST, AND THE ELLIPSE

```
SMOOTH_CASE = FALSE
PressureInitialConditions {
   p0_smooth = (mu*PI/(3*w))*(R^2 - (R+w)^3/R)
   p0_sharp = mu*PI*R
   function = "(sqrt((X0-0.5)^2 + (X1-0.5)^2) < R ? (p0_sharp - mu/(R+w)) : ...)"
}
```

- `ex0`: “This is a pressurized shell, the colormap shows the pressure. There is no significant fluid motion” (2020, slide 4). The input sets the *exact* pressure jump of the ring in equilibrium as the initial pressure (two formulas, for a tension spread smoothly or sharply across w); the fluid should then stay at rest, and `volume.curve` should stay flat. It is the first test of Griffith and Luo’s paper.
- `ex1`: $\gamma = 0.15$ makes the ring an ellipse (0.25 by 0.40 in radius) in a fluid of viscosity 10^{-2}; it relaxes toward a circle and overshoots. “This example comes from Figure 4, Griffith and Luo: representative results from the dynamic ($\gamma = 0.15$) version of the idealized anisotropic shell model of Section 5.1.1 for $N = 128$ over $0 \le t \le 0.75$” (slide 6). As shipped it stops at $t = 0.2$; the longer run (slide 29) reaches $t = 3.9$.

ex2, ex3, ex4: soft bodies pushed by boundary conditions

- ex2. A 2×1 box with no periodic sides and a plate 0.1 thick spanning its full height at $x = 1$ (`build_square` from 0.95 to 1.05); neo-Hookean $\mu = 10$ (the `mu*FF` and `-mu*FF^{-T}` pair of chapter 19); no tether. The x-velocity's boundary condition on the left and right faces is a *traction* ($a = 0$, $b = 1$): -0.1 on the left, $+0.1$ on the right, a normal-stress difference of 0.2 across the box that pushes fluid, and the plate with it. The 2020 reading, "porous FE boundaries ... The porous plate takes up the entire channel, and flow is driven through it", is half right: the plate takes up the channel and the boundary conditions drive the flow, but the code has no porosity, and what goes through the plate goes through because the IB kernel lets the grid's fluid leak through a soft body held by nothing: the test is of that leak. Three levels, $\mathrm{DX} = 1/64$. Also in 3D.
- ex3. "Cavity driven flow with a flexible material on the bottom" (slide 8): a 2×0.5 block on the floor of a 2×2 cavity, `QUAD9`, $c_1 = 0.05$ ($\mathbb{P} = 2c_1\mathbb{F}$, the neo-Hookean form written with $c_1 = \mu_s/2$), the lid moving at 0.5 with $\sin^2$ ramps to zero over 0.3 at the two corners so that the corner singularity of the classic cavity is avoided. Two levels, $\mathrm{DX} = 1/32$.
- ex4. "A soft neo-Hookean disc in a lid-driven cavity flow using the partitioned (split) weak formulation with $N = 128$ and $\mathrm{Mfac} = 4$ over $3 \le t \le 7$" (slide 9, from Griffith and Luo's Fig. 10): a disc of radius 0.2 built by `build_sphere` with `TRI6`, its boundary nodes snapped onto the exact circle, mapped to $(0.6, 0.5)$; lid speed 1; $c_1 = 0.05$ and a dilatational stabilization $\beta_s = \mathrm{NFINEST}/64$ that grows with the grid; `volume.curve`. The shipped input is $N = 64$, one level, one hundred steps ($t = 0.1$): "This example only runs for two data visualization writes. You can increase the endtime to run for longer" (slide 10). Also in 3D (a sphere).

EX5 (1): THE CYLINDER AT $\mathrm{Re} = 200$, AS A SURFACE OR A SOLID

```
BoundaryMesh boundary_mesh(solid_mesh.comm(), solid_mesh.mesh_dimension() - 1);
solid_mesh.boundary_info->sync(boundary_mesh);
boundary_mesh.prepare_for_use();
bool use_boundary_mesh = input_db->getBoolWithDefault("USE_BOUNDARY_MESH", false);
Mesh& mesh = use_boundary_mesh ? boundary_mesh : solid_mesh;
...   // IBFESurfaceMethod for the boundary mesh, IBFEMethod for the solid
F = tether_data->kappa_s_surface * D - tether_data->eta_s_surface * U;   // the surface tether
PP = 2.0 * tether_data->c1_s * (FF - tensor_inverse_transpose(FF, NDIM));  // the solid's penalty stress
```

```
Re = 200.0
USE_BOUNDARY_MESH = TRUE
MAX_LEVELS = 6   N = 16   L = 16.0   MFAC = 8.0   ELEM_TYPE = "TRI6"
KAPPA_S_SURFACE = 1.0e6   ETA_S_SURFACE = 0.0   C1_S = 1.0e5
VelocityBcCoefs_0 { t_half = 0.5  tau = 0.125
   gcoef_function_0 = "(tanh((t-t_half)/tau)+tanh(t_half/tau))/(1+tanh(t_half/tau))" }
VelocityBcCoefs_1 { gcoef_function_0 = "cos(pi*X_1/16.0)*exp(-2.0*t)" }
```

- “From Figure 12, Griffith and Luo: vortices shed from a stationary circular cylinder at $\mathrm{Re} = 200$” (slide 11). Radius 0.5 in a box $-8 < x < 24$, $-8 < y < 8$; six levels on a 16-cell base ($\mathrm{DX} = 1/32$ at the cylinder); the inflow ramped by the same $\tanh$ as chapter 22’s crab, plus a transverse velocity $\cos(\pi y/16)\, e^{-2t}$ at the inflow face that dies in a few time units: the nudge that starts the shedding on a symmetric grid.

EX5 (2): TWO BODIES IN ONE FILE, AND WHAT HAPPENED

- Two bodies in one file. `USE_BOUNDARY_MESH = TRUE` (the shipped choice): the cylinder is its boundary curve only, extracted from the disc by libMesh's `BoundaryMesh`, handled by `IBFESurfaceMethod`, and held by a surface tether $\kappa_s(\mathbf{X}_0 - \mathbf{X}) - \eta_s\mathbf{U}$. `FALSE`: the solid disc with `IBFEMethod`, a body tether and a penalty stress $c_1 = 10^5$. Chapter 21's parts, chapter 19's surface forces, and a thin-surface method this series has not needed.
- Output: `C_D.curve`, `C_L.curve`, `U_max.curve`. "I had some issues reading the .ex2 file for this. It looks like ex11 is a very similar example which does run well" (slide 11). Also in 3D (a sphere).
- What happened. As shipped, in 2D and in 3D, the run aborts after five steps with `Time step size change encountered`; with `DT` halved, after eleven. Slide 24 follows this up: the step is not the problem, the surface tether is.

EX6 (1): THE TUREK–HRON BEAM BEHIND A CYLINDER

```
L     = 2.46          // channel length
H     = 0.41          // channel height
RHO   = 1.0e3
NU    = 1.0e-3
MU    = RHO*NU
U_BAR = 2.0           // mean inflow speed
D     = 0.1           // cylinder diameter
RE    = U_BAR*D*RHO/MU          // 200
MU_S = 2.0e6          // the beam's shear modulus
NU_S = 0.4
LAMBDA_S = 2.0*MU_S*NU_S/(1.0 - 2.0*NU_S)
KAPPA_S = 2*2.5e4 * DX / DT^2   // the block's tether
VelocityBcCoefs_0 { U_bar = U_BAR  H = H
   gcoef_function_0 = "1.5*U_bar*X1*(H-X1)/((H/2)^2)*(t < 2.0 ?
  0.5*(1.0-cos(0.5*pi*t)) : 1.0)" }
```

```
MeshTools::Generation::build_sphere(block_mesh, R, r, ...);
  // then shifted to (0.2, 0.2)
MeshTools::Generation::build_square(beam_mesh, ceil(0.4 / ds),
  ceil(0.02 / ds), 0.2, 0.6, 0.19, 0.21, ...);
vector<MeshBase*> meshes(2);  meshes[0] = &block_mesh;  meshes
  [1] = &beam_mesh;
// the beam's tether holds only its part inside the disc:
const double r = sqrt((s(0) - 0.2) * (s(0) - 0.2) + (s(1) -
  0.2) * (s(1) - 0.2));
if (r <= 0.05) F = elasticity_data->kappa_s * (s - X); else F.
  zero();
```

The input is in SI units, the only example of the twelve that is: a channel 2.46 m long, water-like ρ and ν, a mean inflow of 2 m/s, and a beam of shear modulus 2 MPa. Chapter 8a's rule (regenerate the stiffnesses when the grid or the step changes) is built into `KAPPA_S`.

EX6 (2): THE BENCHMARK, THE TWO PARTS, AND WHAT HAPPENED

- "A standard example of a relatively rigid circle attached to a flexible plate that bends as vortices are shed. I believe it comes from a paper by Boyce, but I have not found it yet" (slide 12). It is the Turek–Hron FSI benchmark (2006), case FSI3: channel 2.5×0.41 (here 2.46), cylinder of diameter 0.1 at $(0.2, 0.2)$, beam 0.35×0.02 behind it, $\rho_f = \rho_s = 10^3$, $\nu = 10^{-3}$, $\bar{U} = 2$ m/s, $\mu_s = 2 \times 10^6$ Pa, $\nu_s = 0.4$, the inflow parabola ramped over 2 s with $\frac{1}{2}(1 - \cos \frac{\pi t}{2})$: every number is the benchmark's.
- Two parts (chapter 21): the disc, held rigid by a tether and a penalty stress; the beam, St. Venant–Kirchhoff with μ_s, λ_s, built 0.4 long so that its root sits inside the disc, where the same tether clamps it. The `ctx` pointer carries the constants into the callbacks (chapter 19).
- Output: the beam tip A at $(0.6, 0.2)$ in `A_x.curve`, `A_y.curve`, and C_D, C_L. FSI3's published tip oscillation: amplitude about 0.034 in y at 5.3 Hz. As shipped: four levels on an 8-cell base ($\mathrm{DX} = 0.038$), one hundred steps.
- What happened. As shipped the run aborts after fourteen steps ($t = 0.004$ s, before the inflow ramp has reached a hundredth of its value) with `Time step size change encountered`; slide 24.

EX8 (1): A CLAMPED BEAM OVER A SLOT

```
build_square(block1_mesh, ..., 0.0, 0.5, 0.0, 0.5, ...);   // two rigid blocks
build_square(block2_mesh, ..., 1.5, 2.0, 0.0, 0.5, ...);
const double beam_x_lower = 0.5, beam_x_upper = 1.5;     // the beam spans the slot
const double beam_y_lower = 0.5 - 0.016;                 // 0.032 thick
build_square(beam_mesh, n_beam_x, n_beam_y, beam_x_lower, beam_x_upper, beam_y_lower, beam_y_upper, QUAD4);
if (beam_use_mapped_grid) { ... cheby(...) ... }         // Chebyshev-spaced nodes, finer near the ends
// the beam's tether acts on its two ends only (a surface force on sides 1 and 3):
if (side == 1 || side == 3) F = beam_kappa_s * (s - X); else F.zero();
PP = mu_s * (FF - FF_inv_trans);  // plus a beta_s log(J) term when BETA_S != 0
```

```
MU_S = 1.0e3   P_MAX = 80.0   T_END = 2.0
BETA_S = SAFETY * 1.25e-2 * MU_S / sqrt(DT)
BEAM_KAPPA_S = SAFETY * 5.0e1 * DX / DT^2
VelocityBcCoefs_1 {   // the bottom face, between the blocks: a pressure, ramped
   acoef_function_2 = "(X_0 > 0.5) && (X_0 < 1.5) ? 0.0 : 1.0"
   bcoef_function_2 = "(X_0 > 0.5) && (X_0 < 1.5) ? 1.0 : 0.0"
   gcoef_function_2 = "(X_0 > 0.5) && (X_0 < 1.5) ? -p_max * min(1.0, t/t_end) : 0.0" }
```

- The boundary condition on the bottom face is written per point: Dirichlet ($a = 1$) outside the slot, a normal traction ($b = 1$) that ramps to -80 inside it. Chapter 22's recipe for a mixed face in one expression.

EX8 (2): WHAT IT IS, AND WHAT TO COPY

- "The second example of possible porous boundaries in IBAMR. The example consists of two solid blocks with a rigid, porous plate on top. Flow is driven from the bottom of the domain through the porous plate" (slide 14). The code: two blocks 0.5×0.5 held rigid by body tethers, a beam 1 long and 0.032 thick spanning the slot between them, clamped at its two ends by a *surface* tether (chapter 19, slide 24; chapter 21's ex8 pattern), and a pressure of 80 applied on the bottom face under the slot, ramped over 2 s: the fluid is pushed up through the slot and bows the beam. Neither rigid nor porous: an elastic beam, $\mu_s = 10^3$, nearly incompressible, with a stabilization $\beta_s \propto \mu_s/\sqrt{\mathrm{DT}}$.
- Two things to copy: the Chebyshev spacing of the beam's nodes (finer at the clamped ends, where the bending is), and the post-processing: the beam centre line's deformed length and the flux through the slot (`compute_deformed_length`, `compute_inflow_flux`), the latter a loop over the patches touching the bottom face.
- Shipped: $N = 48$, one level, one hundred steps ($t = 0.04$, before the ramp has done anything); the longer run (slide 29) shows the bow.

EX10 AND EX11 (1): RIGID BODIES BY DIRECT FORCING

```
#include <ibamr/IBFEDirectForcingKinematics.h>
void cylinder_kinematics(double /*data_time*/, Eigen::Vector3d& U_com, Eigen::Vector3d& W_com, void* /*ctx*/)
{   U_com.setZero();  W_com.setZero(); }        // the prescribed part of the motion: none
...
Pointer<IBFEDirectForcingKinematics> df_kinematics_ops = new IBFEDirectForcingKinematics(
    "cylinder_dfk", app_initializer->getComponentDatabase("CylinderIBFEDirectForcingKinematics"), ib_method_ops, /*part*/ 0);
ib_method_ops->registerDirectForcingKinematics(df_kinematics_ops, /*part*/ 0);
df_kinematics_ops->setSolveRigidBodyVelocity(solve_dofs);     // ex10: which of the body's six degrees of freedom are free
df_kinematics_ops->registerKinematicsFunction(&cylinder_kinematics, nullptr);
```

- The mechanism. `IBFEDirectForcingKinematics` makes a part rigid without tethers: each step it computes the fluid force on the body, advances the body's rigid translation and rotation (for the degrees of freedom that are free), and then forces the body's velocity onto the fluid. The kinematics function supplies the *prescribed* part (zero here); `setSolveRigidBodyVelocity` says which components are solved for. The part's finite element mesh is only a shape.
- Compare chapter 14's constraint IB (the same idea for point bodies: compute the force, move the body, impose its velocity) and chapter 20's targets (a soft version: the body is pulled, not set).

EX10 (2): A FREE CYLINDER IN A COUETTE FLOW

```
R = 0.5   D = 2*R   H = 4*D   L = 40*H   Uw = 1.0   Re = 40.0
MU = 2.0*RHO*Uw*H/Re
CylinderIBFEDirectForcingKinematics { rho_s = RHO }   // neutrally buoyant
VelocityBcCoefs_0 { gcoef_function_2 = "U_wall"  gcoef_function_3 = "-U_wall" }
periodic_dimension = 1,0
```

- ex10 (README: “FSI of cylinder in a shear-driven channel flow at $\mathrm{Re} = 40$”): a neutrally buoyant cylinder of diameter 1 in a channel 4 high and 160 long, periodic in x, with the walls moving at $+1$ and -1: a plane Couette flow. The cylinder is free, and its centre’s path (`X.curve`) and velocity (`U.curve`) show it migrate to the centre line, the classic inertial-migration result for a free body in a shear flow. “This is a long simulation with many file writes” (slide 15): `END_TIME = 1200`.
- What happened. On four cores the shipped run reaches $t = 28$ in fifteen minutes (slide 28); `X.curve` and `U.curve` have the centre’s path to there. The migration takes hundreds of time units: exercise 6.

ex11 (3): the fixed cylinder at $\mathrm{Re} = 200$ by direct forcing

```
Re = 200.0   N = 256   MAX_LEVELS = 2   REF_RATIO = 4   DT = 0.002   END_TIME = 130.0
x_lo = -8.0, -8.0   x_up = 24.0, 8.0
```

- `ex11` (README: "flow past cylinder using IBFEDirectForcingKinematics at $\mathrm{Re} = 200$"): the fixed-cylinder benchmark of `ex5` done with a rigid body instead of tethers, on a 256×128 base with one refinement; C_D, C_L to `.curve` files and `compute_St.m`, which finds the Strouhal number from the lift by a Fourier transform and by zero crossings (published: $\mathrm{St} \approx 0.19$–0.20, $C_D \approx 1.3$–1.4 at $\mathrm{Re} = 200$). `END_TIME = 130`.
- What happened. It runs as shipped, and fast: the whole benchmark, $65\,000$ steps to $t = 130$, took 27 minutes on four cores (slide 29), writing 651 frames; the gallery shows its wake. "Long" in 2020 was the course cluster's queue, not the problem. The `ex5` ".ex2 read issue" of 2020 that sent the lecturer here does not arise with VisIt 3.5.

ex9, and the ones that stop as shipped

- ex9 has no 2020 slide. Its code: a cylinder's *boundary mesh* (libMesh's `BoundaryMesh` of a disc, one-dimensional elements in a two-dimensional box) is the IB body, tethered to two fields defined on the *solid* disc, a "position" and a "velocity" system, with $\mathbf{F} = \kappa_s(\mathbf{x}_{\text{solid}} - \mathbf{x}) + \eta_s(\mathbf{u}_{\text{solid}} - \mathbf{u})$; the callback finds the solid element under each boundary element with `interior_parent()`. As shipped the position field is the rest shape and the velocity zero, the box (16×32, traction-free sides, no inflow) is still, and nothing happens: the file is a template for a surface that follows a motion defined on a volume, with both a position and a velocity penalty, not a flow problem. Its `// \todo` says so: "There needs to be an API so that this can be handled by the IBFEMethod class implementation."
- Why it stops now. Built and run against 0.19.0 it aborts before the first step:

```
Failed assertion:  mesh->mesh_dimension() == mesh->spatial_dimension()   (IBFEMethod.cpp, line 376)
```

- `IBFEMethod` now accepts only meshes of the box's dimension (areas in 2D, volumes in 3D); a curve in 2D or a surface in 3D goes to `IBFESurfaceMethod` (what `ex5` switches to for its boundary mesh) or to the `IIM` suite. Chapter 15 met the same assertion in `IBLevelSet/ex0`. `ex9`'s 3D input stops the same way.
- ex5 (2D and 3D) and ex6 stop after five and fourteen steps with the other classic message, `Time step size change encountered`: slide 24.

“TIME STEP SIZE CHANGE ENCOUNTERED” (1): WHAT THE MESSAGE MEANS

- The message comes from `IBHierarchyIntegrator::preprocessIntegrateHierarchy()` (line 213 in 0.19.0) when `error_on_dt_change = TRUE` (every example input sets it through `ERROR_ON_DT_CHANGE`) and the step the integrator wants differs from the last one: the CFL limit, computed from the current velocity, has fallen below `dt_max`. The abort is deliberate: a shrinking step means the velocity has grown past what the input assumed, usually the start of an instability, and the examples would rather stop than run on with a step that changes. With `error_on_dt_change = FALSE` and `warn_on_dt_change = TRUE` the same check prints “WARNING: Time step size change encountered. Suggest reducing maximum time step size in input file” and goes on with the smaller step.
- “Some of the examples go unstable (timestep size change encountered). Just reduce DT in the input file and rerun” (2020, slide 3). Two of the twelve abort as shipped, `ex5` after five steps (2D and 3D) and `ex6` after fourteen, before either inflow ramp has done anything: whatever grows, grows from rest. The next slide tries the tip and two other things on each.
- What the step must satisfy here: the fluid CFL ($U\,\mathrm{DT}/\mathrm{DX} \le$ `cfl`); a penalty spring’s limit, $\mathrm{DT} \lesssim \sqrt{m/\kappa_s}$ for the mass m a spring moves (chapter 8a’s rule: the inputs write $\kappa_s \propto \mathrm{DX}/\mathrm{DT}^2$ *from* the step, so halving `DT` quadruples κ_s and keeps the ratio); and the elastic wave speed $\sqrt{\mu_s/\rho_s}$ in a stiff body (chapter 16). An input that fixes κ_s by hand, as `ex5` does, breaks the first rule when the grid or the step changes.

"TIME STEP SIZE CHANGE ENCOUNTERED" (2): WHAT WAS TRIED ON EX5 AND EX6

	`ex5` (a cylinder surface held by `KAPPA_S_SURFACE = 1e6`, `ETA = 0`)	`ex6` (a disc held by `KAPPA_S = 2*2.5e4*DX/DT^2`, a beam of $\mu_s = 2$ MPa)
as shipped	aborts at step 5	aborts at step 14
`DT` halved (the 2020 tip)	aborts at step 11: the tip only delays it	runs the hundred steps: the tip works
the step allowed to change	finishes the shipped run in 1585 steps instead of 100: the step falls to a tenth to a thirtieth of `DT` and stays there, fluctuating	finishes in 403 steps: the step settles at about a quarter of `DT`
the tether changed	the input's own, unused, grid-scaled constants (`KAPPA_S = 2.5*DX/DT^2`, `ETA_S = 1.5*DX/DT`, lines 48–49 of `input2d`) in place of the fixed 10^6 and 0: runs the hundred steps at the shipped `DT`, in 6 s	a hundred times softer (`2*2.5e2`): aborts at step 17
reading	the undamped surface spring is thirty times stiffer than the grid-scaled rule allows at this step; the surface rings and the fluid next to it moves at tens of units while the inflow is still zero	the step, not the tether: at the shipped `DT` the beam's elastic wave ($\sqrt{\mu_s/\rho_s} \approx 45$ m/s) crosses a third of a cell per step, and the coupled scheme does not tolerate it; half the step does

The fix for `ex5` is in its own input file; the fix for `ex6` is the 2020 tip. Letting the step change is a diagnosis, not a fix: it shows how far the step wants to fall, at four to sixteen times the cost.

THE MECHANISMS THESE EXAMPLES ADD (1)

mechanism	where	what it is for
a surface (codimension-one) body with `IBFESurfaceMethod`	`ex5`	a thin shell or membrane as a curve or surface mesh, with its own interaction rules; what `IBFEMethod` no longer accepts
a tether that reads fields from another mesh (`interior_parent`, `point_value`)	`ex9`	drive a surface from a motion defined on a volume; a position and a velocity penalty together
rigid-body motion by direct forcing (`IBFEDirectForcingKinematics`)	`ex10`, `ex11`	a rigid part without tethers, free or prescribed, with the fluid force integrated for you
a stabilized nearly incompressible beam ($\beta_s \propto \mu_s/\sqrt{\mathrm{DT}}$) and Chebyshev node spacing	`ex8`	thin elastic structures that must not lose volume

These four are new to this series; the next slide lists the smaller devices worth copying.

THE MECHANISMS THESE EXAMPLES ADD (2): SMALLER DEVICES

device	where	what it is for
a penalty stress $c_1(\mathbb{F} - \mathbb{F}^{-T})$ plus a tether for a "rigid" part	`ex5`, `ex6`	a rigid body done with elasticity, the way Part IV did it
the exact pressure as an initial condition	`ex0`	a static test with a known answer
a transverse perturbation at the inflow that decays	`ex5`, `ex11`	start shedding on a symmetric grid without waiting for round-off
constants through `ctx`	`ex5`, `ex6`	chapter 19's way to give callbacks their parameters

The ones Part IV already used: parts and per-part registration (`ex6`–`ex8`), the coordinate mapping (`ex0`, `ex1`, `ex4`), body and surface tethers (all), two stress functions (`ex2`–`ex4`, `ex7`, `ex8`), the sponge layer (`ex7`).

WHAT WAS RUN FOR THIS CHAPTER

example	dim	input	END_TIME	steps	time t	wall	outcome
ex0	2D	input2d	shipped	100	0.195312	2 s	ran to the end
ex1	2D	input2d	shipped	100	0.195312	3 s	ran to the end
ex2	2D	input2d	shipped	100	0.390625	2 s	ran to the end
ex3	2D	input2d	shipped	100	1.875	2 s	ran to the end
ex4	2D	input2d	shipped	100	0.048828	1 s	ran to the end
ex5	2D	input2d	shipped	5	0.007812	1 s	time-step change abort (slide 24)
ex6	2D	input2d	shipped	14	0.004035	1 s	time-step change abort (slide 24)
ex7	2D	input2d.caseC	shipped	1000	0.019531	33 s	ran to the end
ex8	2D	input2d	shipped	100	0.041666	1 s	ran to the end
ex9	2D	input2d	shipped	0	–	0 s	mesh-dimension assertion (slide 23)
ex10	2D	input2d	shipped	5585	27.925	901 s	stopped at the cap
ex11	2D	input2d	shipped	32544	65.088	901 s	stopped at the cap

The twelve 2D inputs as shipped, built standalone (30–50 s each) and run on four cores. Steps and the time t reached; `END_TIME = 100*DT` for most of them is a hundred steps. Nine of the twelve run as shipped.

WHAT WAS RUN FOR THIS CHAPTER (2): THE LONGER RUNS AND THE 3D INPUTS

example	dim	input	END_TIME	steps	time t	wall	outcome
ex0	2D	`input2d`	2000*DT	2000	3.90625	21 s	ran to the end
ex1	2D	`input2d`	2000*DT	2000	3.90625	47 s	ran to the end
ex2	2D	`input2d`	2000*DT	2000	7.8125	41 s	ran to the end
ex2	3D	`input3d`	shipped	100	0.390625	39 s	ran to the end
ex3	2D	`input2d`	2000*DT	2000	37.5	37 s	ran to the end
ex4	2D	`input2d`	2000*DT	2000	0.976562	21 s	ran to the end
ex4	3D	`input3d`	shipped	100	0.097656	13 s	ran to the end
ex5	2D	`input2d`	2000*DT	5	0.007812	0 s	time-step change abort (slide 24)
ex6	2D	`input2d`	2000*DT	14	0.004035	1 s	time-step change abort (slide 24)
ex8	2D	`input2d`	2000*DT	2000	0.833333	25 s	ran to the end
ex9	2D	`input2d`	shipped	0	–	0 s	mesh-dimension assertion (slide 23)
ex10	2D	`input2d`	shipped	10450	52.25	1801 s	stopped at the cap
ex11	2D	`input2d`	shipped	65000	130	1626 s	ran to the end

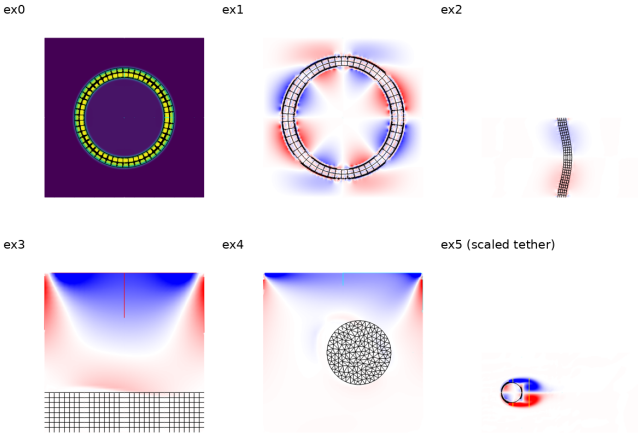
The smoke tests rerun twenty times longer (a 15-minute cap), the benchmarks under a 30-minute cap, and the 3D inputs as shipped. “stopped at the cap” means the wall-clock limit, not an error.

WHAT WAS RUN FOR THIS CHAPTER (3): THE VARIANTS, AND THE COMPLEX-FLUIDS EXAMPLES

example	dim	variant	END_TIME	steps	time t	wall	outcome
ex5	2D	step allowed to change	shipped	1585	0.15625	103 s	ran to the end
ex5	2D	DT halved	shipped	11	0.008593	1 s	time-step change abort (slide 24)
ex5	2D	scaled tether, 2000*DT	2000*DT	2000	3.125	31 s	ran to the end
ex5	2D	the input's scaled tether	shipped	100	0.15625	6 s	ran to the end
ex5	3D	step allowed to change	shipped	88	0.012790	302 s	stopped at the cap
ex5	3D	DT halved	shipped	11	0.008593	3 s	time-step change abort (slide 24)
ex6	2D	step allowed to change	shipped	403	0.028828	16 s	ran to the end
ex6	2D	DT halved	shipped	100	0.014414	3 s	ran to the end
ex6	2D	DT halved, 2000*DT	2000*DT	2000	0.288281	20 s	ran to the end
ex6	2D	tether 100x softer	shipped	17	0.004900	3 s	time-step change abort (slide 24)
cf/ex2	2D	one rank	shipped	111	0.0555	301 s	stopped at the cap
cf/ex2	2D	as shipped	shipped	0	0	1 s	PETSc: no ILU for a parallel matrix (slide 33)
cf/ex4	2D	one rank	shipped	6600	0.66	301 s	stopped at the cap
cf/ex4	2D	as shipped	shipped	0	0	1 s	PETSc: no ILU for a parallel matrix (slide 33)

What was tried on the runs that abort (slide 24): DT halved (the 2020 tip), the step allowed to change (error_on_dt_change = FALSE), and softer tethers; and the two complex-fluids examples with bodies, on four ranks and on one.

GALLERY (1)



Vorticity (red and blue) at the last frame of each run, the bodies' meshes in black; the longer run where one exists; `ex0`, a static test, shows its pressure instead. Left to right, top to bottom: `ex0`, `ex1`, `ex2`, `ex3`, `ex4`, `ex5` (with the input's scaled tether, slide 25).

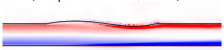
IBAMR 0.19.0, four cores; `wsl/final/visit_ibfe_tour.sh`.

GALLERY (2)

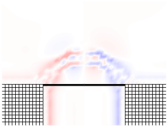
ex6 (DT halved)



ex7 (chapter 23's run, t = 2)



ex8



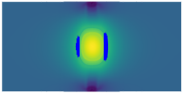
ex10



ex11



ex2-3d (|U| on the mid-plane)



`ex6` (with `DT` halved, $t = 0.29$ s, the cylinder and beam end of the channel), `ex7` (chapter 23's run of the same example at $t = 2$; as shipped it stops at $t = 0.02$), `ex8`, `ex10` (a window round the moving cylinder), `ex11` (at $t = 130$, the end of the benchmark), and `ex2` in 3D ($|\mathbf{u}|$ on the mid-plane slice). `ex9` produced no frame, and the complex-fluids runs' SAMRAI dumps crash VisIt 3.5's metadata server (their `.curve` files are the output to read).

THE COMPLEX-FLUIDS EXAMPLES WITH BODIES

- “These examples come from Aaron Barrett’s dissertation. They are all 2D examples of complex fluids modeled using Oldroyd-B (I think). I could not get the examples with immersed boundaries to work (yet)” (2020, slide 17). The suite is `examples/complex_fluids/ex0–ex4`: a viscoelastic (Oldroyd-B) fluid solver with the polymer stress as an extra transported tensor.
- Without bodies (chapter 15 ran them): `ex0`, pipe flow with an exact solution (“error norms are printed out for the components of the conformation tensor and for the velocity”); `ex1`, a four-roll mill driven by a periodic body force (“as the Weissenberg number increases, singularities in the trace form at extensional points”); `ex3`, a lid-driven cavity with a quartic lid profile.
- With bodies (run for this chapter): `ex2`, “flow past a confined cylinder. The structure is tethered in place using a penalty method. The structure is represented using a finite element representation following the IBFEMethod. Drag coefficients are outputted in text files. An interpolation class is included to interpolate the stress onto the surface of the cylinder” (README); `ex4`, “a four roll mill geometry with the four rollers represented by immersed structures. Rotation is imposed on the structures via a penalty method.”

THE COMPLEX-FLUIDS EXAMPLES WITH BODIES (2): WHY THEY DID NOT WORK, AND HOW THEY RUN

- Why they did not work in 2020, and how they run now. On four ranks both stop before the first step with a PETSc error: `Could not locate a solver type for factorization type ILU and matrix type mpiaij`. One of the viscoelastic solver's preconditioners asks for an incomplete LU factorization, which PETSc provides only for a matrix that lives on one rank (`seqaij`); on a parallel matrix (`mpiaij`) it needs a block-Jacobi wrapper or another preconditioner, and the shipped inputs ask for neither. On one rank (`mpirun -np 1`) both start and run: slide 30 says how far each got in five minutes. The 2020 "(yet)" was almost certainly this: the course ran everything on several ranks.
- The IBFE side of both is Part IV's: a tethered body with `IBFEMethod`; what is new is the fluid, which this series does not cover beyond this slide.

WHERE TO START, FOR A GIVEN PROBLEM

you want	start from
a soft body deformed by a flow, no tether	`ex3` or `ex4` (and this series' IBFE/01)
a rigid body held in a stream, with drag and lift	`ex11` (direct forcing) or `ex5` with the solid mesh (tethers); this series' IBFE/10 for the tether version in 3D
a free rigid body	`ex10`
an elastic beam clamped to a rigid part	`ex6` (behind a cylinder) or `ex8` (over a slot); this series' IBFE/08 for two free parts
a thin membrane or shell as a surface	`ex5` with `USE_BOUNDARY_MESH = TRUE`, then the `IIM` suite
a body whose motion is prescribed	chapter 20's targets (volume meshes); `ex9`'s pattern once ported to `IBFESurfaceMethod` (surfaces)
a benchmark to validate against	`ex6` (Turek–Hron FSI3), `ex11` (cylinder at $\mathrm{Re} = 200$), `ex0` (exact pressure jump)
a scalar in the flow	this series' IBFE/12 (chapter 24), `IB/explicit/ex7`
a viscoelastic fluid round a body	`complex_fluids/ex2`

Chapter 23's recipe reads any of them: `ModelData`, the `register` calls, the `getComponentDatabase` names, the mesh, the output block.

WHAT CAN GO WRONG

symptom	cause and fix
`Failed assertion: mesh->mesh_dimension() == mesh->spatial_dimension()`	a curve or surface mesh given to `IBFEMethod` (`ex9`); use `IBFESurfaceMethod` or `IIM`, or a volume mesh
`Time step size change encountered`	the CFL step fell below `dt_max`: a tether too stiff for the step (`ex5`: use the input's grid-scaled `KAPPA_S`) or a step too long for the body (`ex6`: halve `DT`); let the step change only to see how far it falls (slide 25)
a hundred steps and the run ends	`END_TIME = 100*DT`: a smoke test as shipped; raise it
the in-tree `CMakeLists.txt` fails outside the tree	it calls `IBAMR_ADD_EXAMPLE`; replace it with the six lines (slide 9)
`convergence_tester.cpp` will not link	it has its own `main`; leave it out of the executable
`ex7` wants `input2d`	it ships three cases; name one on the command line
the ExodusII file will not open in VisIt (`ex5`, 2020)	try the current build and VisIt 3.5 before blaming the file; `ex5` with the boundary mesh writes the surface's file
no picture in the gallery for a run	the run wrote no frames before its cap, or stopped (the table says which)
`ex10` takes hours	$240\,000$ steps: half an hour on four cores reaches $t = 52$ of 1200, so the whole run is about twelve hours; `ex11`'s $65\,000$ steps take 27 minutes
`complex_fluids/ex2`, `ex4`: `PETSC ERROR ... ILU and matrix type mpiaij`	run them on one rank, or give the solver a parallel preconditioner (slide 33)
VisIt's metadata server crashes on a `dumps.visit`	seen here only for the complex-fluids runs (the extra tensor fields); read their `.curve` files, or try a newer VisIt

WHAT THE 2020 DECK HAD THAT IS NOW PART X

2020	now
“move a Makefile and script.sh to each folder”	the six-line `CMakeLists.txt`; `mpirun` on the laptop, `sbatch` on Puma (chapter 31)
“Make sure the Makefile has the right dimensions”	the CMake target `IBAMR2d` or `IBAMR3d`
“Drop the .C / .cpp and then make”	list only `example.cpp` in `add_executable`
`ex2` and `ex8` as “porous” plates	a soft plate pushed by tractions; an elastic beam over a slot
`ex6` “from a paper by Boyce”	Turek and Hron (2006), FSI3
`ex7` “peristaltic transport”	the collapsible channel of chapter 23
the `ex5` “.ex2 read issue”	not reproduced with 0.19.0 and VisIt 3.5 (run table)
no slide for `ex9`; nothing after `ex11`	slide 23; there is nothing after `ex11` in 0.19.0
complex-fluid IB examples “not working (yet)”	they need one rank (PETSc’s ILU); on one rank they run (slide 33)
three figure crops from Griffith and Luo	own renders (gallery)

EXERCISES

1. Run `ex1` to $t = 0.75$ and plot the ring's area from `volume.curve`; compare the shape at $t = 0.75$ with Griffith and Luo's Fig. 4.
2. Run `ex6` with `DT` halved to $t = 10$ (about $70\,000$ steps, half an hour on four cores at the shipped grid) and plot `A_y.curve`: the FSI3 tip oscillation is about ± 0.034 at 5.3 Hz once periodic.
3. `ex11` for 30 time units, then `compute_St.m` (or its Python translation, chapter 33) on `C_L.curve`; compare St and the mean C_D with the published values for $\mathrm{Re} = 200$.
4. `ex5` with `USE_BOUNDARY_MESH = FALSE`: does the solid disc give the same C_D as the surface?
5. `ex8`: plot the beam's deformed length against the pressure ramp; then double `MU_S`.
6. `ex10` to $t = 100$: where is the cylinder's centre, and how fast is it still moving?
7. Port `ex9` to `IBFESurfaceMethod` (the way `ex5` does it) and give its velocity field a rotation.
8. `ex5` in 3D: does the grid-scaled tether rescue it as it does in 2D (slide 25)? Then compare its C_D with the 2D value.
9. `ex6` at the shipped `DT` with `MU_S` ten times smaller: does it run? What does that say about the limit?

RULES OF THUMB

- Read the code before the lecture note: “I believe” marks a guess, and the code settles it.
- As shipped, most IBFE examples are hundred-step smoke tests; the benchmarks are half-hour jobs on a desktop PC (`ex11`, `ex6` to a few seconds) to half a day (`ex10`).
- A curve in 2D or a surface in 3D is not an `IBFEMethod` body any more.
- “Time step size change” means the velocity outgrew the input: a tether too stiff or a step too long. Grid-scaled constants first, then a smaller `DT`; let the step adapt only to diagnose.
- Rigid parts: direct forcing when you want the force and the free motion, tethers when you want control.
- Every example writes `.curve` files; read them before the pictures.

SUMMARY

1. IBAMR's twelve IBFE examples: a pressurized ring and its elliptical start, three soft bodies pushed by boundary conditions, a cylinder at $\mathrm{Re} = 200$ as a surface or a solid, the Turek–Hron beam, the collapsible channel, a clamped beam over a slot, a surface tethered to a volume's motion, and two rigid cylinders by direct forcing.
2. They add four mechanisms to Part IV's: surface meshes with their own method, tethers that read another mesh, direct forcing for rigid bodies, and a stabilized beam.
3. All were built standalone and run: nine run as shipped; `ex9` stops on the mesh-dimension assertion; `ex5` and `ex6` stop on the time-step abort, and run again with the input's own grid-scaled tether (`ex5`) or `DT` halved (`ex6`); the run tables give steps, times and costs.
4. Chapter 15's one-line map was wrong and is corrected; the 2020 readings of `ex2`, `ex6`, `ex7` and `ex8` are corrected from the code.
5. For a new problem, slide 35 says which example to start from and which chapter explains it.

Part IV's IBFE chapters are complete. Next: Part V, VisIt. (The horse's carotid artery, planned here as an IBFE example, became IB/13, a 2D IB example: chapter 36.)

RECORDINGS (LISTED, NOT INCORPORATED)

The 2020 Zoom recording of *6 – IBFE – What do Boyce's examples do?* is listed in the front matter's video list (chapter 0.1). The deck's three figure crops from Griffith and Luo (2017) are not reproduced; the gallery shows this chapter's runs instead.

REFERENCES AND CREDITS

- IBAMR 0.19.0: `examples/IBFE/explicit/ex0`–`ex11` and `examples/complex_fluids/ex0`–`ex4`, their inputs and READMEs; `src/IB/IBFEMethod.cpp`. https://github.com/IBAMR/IBAMR
- B. E. Griffith and X. Luo, *Hybrid finite difference/finite element immersed boundary method*, Int. J. Numer. Meth. Biomed. Engng 33 (2017) e2888: the IBFE method and the tests `ex0`, `ex1`, `ex4` and `ex5` reproduce (its Figs. 4, 10, 12).
- S. Turek and J. Hron, *Proposal for numerical benchmarking of fluid–structure interaction between an elastic object and laminar incompressible flow*, in H.-J. Bungartz and M. Schäfer (eds.), Fluid–Structure Interaction, Springer (2006) 371–385: `ex6`.
- A. P. S. Bhalla, R. Bale, B. E. Griffith and N. A. Patankar, *A unified mathematical framework and an adaptive numerical method for fluid–structure interaction with rigid, deforming, and elastic bodies*, J. Comput. Phys. 250 (2013) 446–476: the direct-forcing idea behind `ex10` and `ex11`.
- The course material: *6 – IBFE – What do Boyce's examples do?* (2020). The runs: `/root/work/0.19.0/ibfe_tour/` (IBAMR 0.19.0, four cores); figures with VisIt 3.5.